



Module 5: Scheduling Basic Jobs

Slides & Exercises

https://github.com/ResearchComputing/hpc_fundamentals_micro_credential

- In “scheduling_jobs” directory



Learning Objectives

- Obtain a clear overview of job scheduling on an HPC system
- Learn to schedule both **batch** and **interactive** jobs on Alpine using the terminal

Session Overview

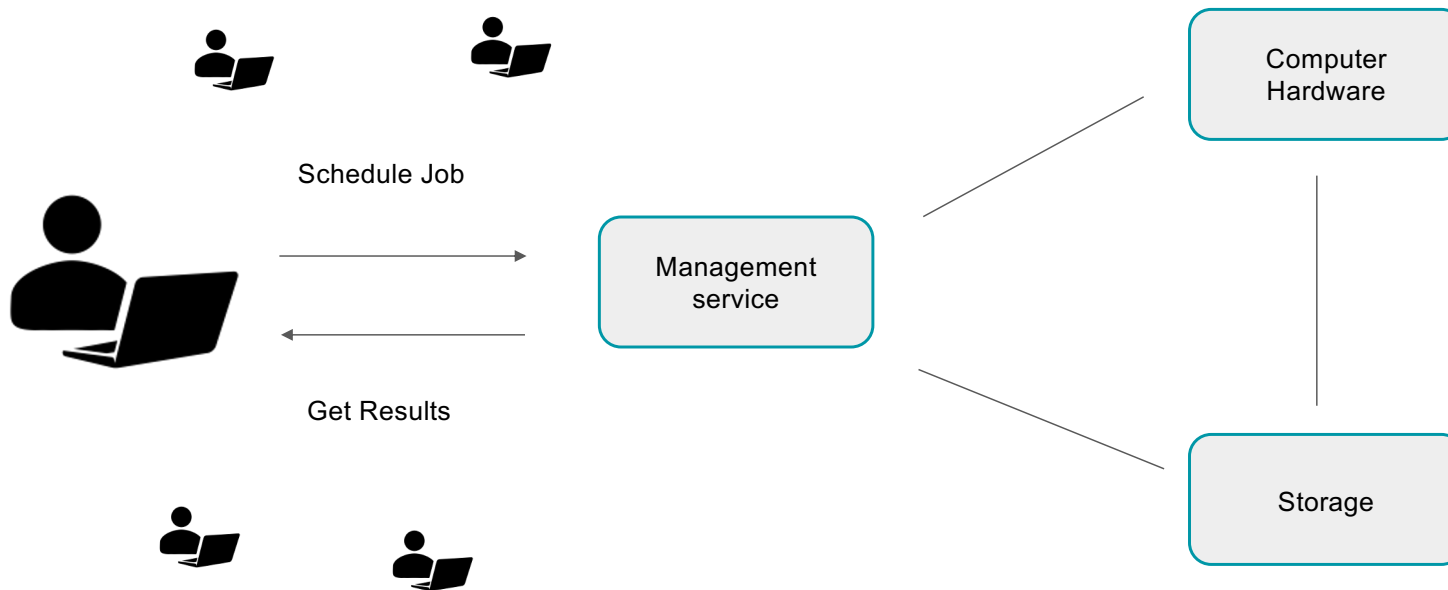
- General overview of job scheduling
- Introduction to batch jobs
 - Scheduling a batch job
- Checking/monitoring jobs
- Utilizing software in a job
- Introduction to interactive jobs

Reminder!

When you login to the HPC system you are on a ***login*** node.

You then need to schedule a job on a ***compute*** node to run your workflow.

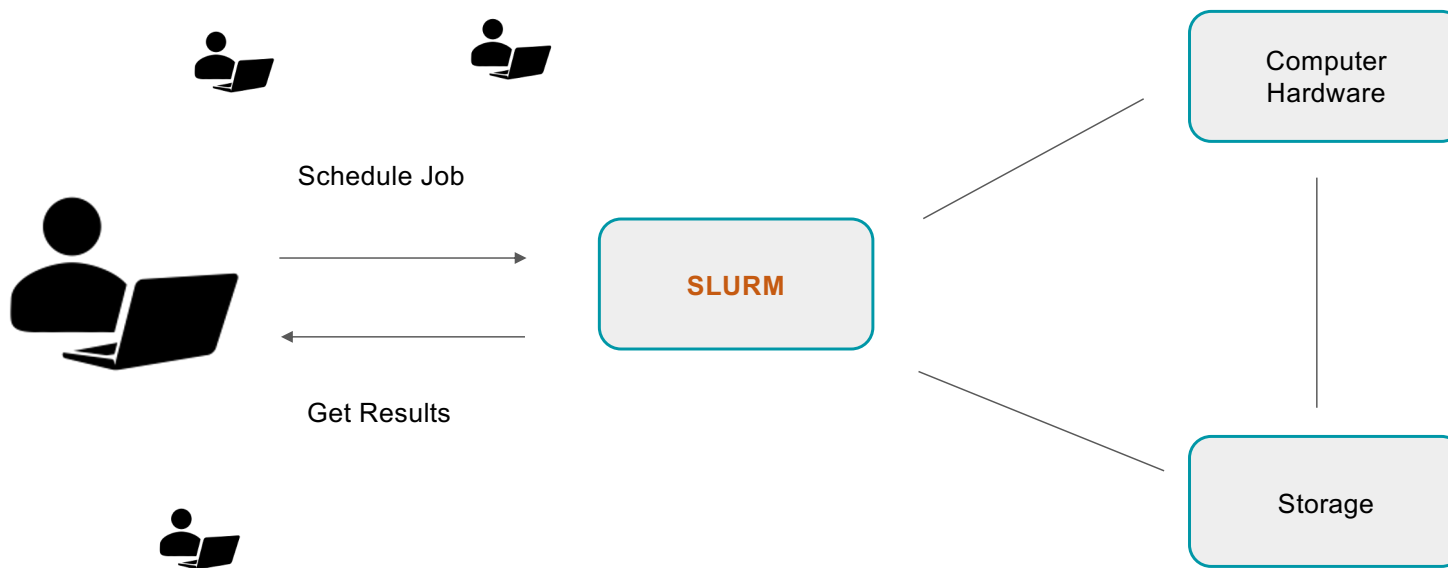
General Overview of Job Scheduling



Introduction to Jobs

- Because HPC clusters are shared resources with many users running their applications, we need software to manage and allocate resources on *compute* nodes.
- **SLURM**
 - Simple **L**inux **U**tility for **R**esource **M**anagement
- Through SLURM, users can obtain allotments of computing resources called **jobs**
- 2 Types of jobs
 - **Batch jobs**
 - **Interactive jobs**

General Overview of Job Scheduling



Job directives

- Directives are special flags that specify what type of HPC resources you would like to use
 - *“I want to run on 1 CPU for 1 hour”*
- Used for both batch and interactive jobs
- Can be specified via the command line or in a job script
- All directives shown are specific to SLURM

Common directives (mandatory)

- ★ • Partition – A collection of compute nodes with similar characteristics
 - `--partition=<partition_name>`
- ★ • Quality of service (QoS) – System defined constraints for a job (more on this later!)
 - `--qos=<qos>`
- ★ • Number of nodes to run on
 - `--nodes=<nodes>`
- ★ • Number of CPU cores to run on
 - `--ntasks=<number-of-tasks>`
- ★ • Wall time – How long you want to run on these resources
 - `--time=<wall time>`

★ - Must be specified for every job

Common directives (optional)

- **Allocation – Account to “charge to”**
 - `--account=<account_name>`
- **Job name**
 - `--job-name=<jobname>`
- **Output – Where all output that would be written to the terminal should go**
 - `--output=<name>`
- **Send an email when events happen in the job**
 - `--mail-type=<event_type>`
- **Email address to send updates to**
 - `--mail-user=<user>`

Alpine Partitions

Partition	Description	# nodes	RAM/core (GB)	cores/node	GPUs/node
acpu	General Compute Node	<u>420</u>	3.75	32 to 128	0
amem	High-memory Node	24	<u>16</u>	48-128	0
ami100	GPU Node: AMD MI100	8	3.75	64	<u>3</u>
aa100	GPU Node: Nvidia A100	12	3.75	64	<u>3</u>
al40	GPU Node: Nvidia L40	3	3.75	64	<u>3</u>
ah200	GPU Node: Nvidia H200	8	11.2	128	<u>4</u>
artxpro6000	GPU Node: Nvidia RTX Pro 6000	8	11.2	128	<u>4</u>

Quality of Service (QoS)

QoS specifies additional constraints for a job

QoS name	Description	Max walltime	Max jobs/user	Max hardware/user	Valid Partitions
cpu-normal	Standard QoS for CPU jobs	1 day	1000	128 nodes	acpu
cpu-long	Longer CPU jobs	7 days	200	20 nodes	acpu
mem-normal	Standard QoS for High-memory jobs	24 hours	1000	256 CPU cores	amem
mem-long	Longer mem jobs	7 days	200	185 CPU cores	amem
gpu-normal	Standard QoS for GPU jobs	24 hours	1000	see Available GRES on Alpine	aa100,ami100,al40,ah200,artxpro6000
gpu-long	Longer GPU jobs	7 days	200	see Available GRES on Alpine	aa100,ami100,al40,ah200,artxpro6000

This list is not exhaustive. Information for other QoS's (e.g., for testing), can be found in the [Alpine QoS documentation](#).

Batch Jobs

- **Batch Jobs** are jobs you submit to the scheduler that are run later without supervision
 - By far the most common job type on Alpine
 - Requires a job script
- A job script is simply a script that includes **SLURM directives** (resource specifics) ahead of any commands.

Anatomy of a job script

- It's just a bash script with SLURM specific directives!

```
#!/bin/bash

## Directives
#SBATCH --<option>=<value>

## Software
module load <software>

## User scripting
<command>
```

Directives in a job script

```
#SBATCH --<option>=<value>
```



Example job script

```
#!/bin/bash

## Directives
#SBATCH --nodes=1                # Number of requested nodes
#SBATCH --ntasks=1              # Number of requested tasks/cores
#SBATCH --time=0:01:00          # Max run time
#SBATCH --partition=acpu         # Specify Alpine CPU node
#SBATCH --qos=cpu-normal        # Specify QoS
#SBATCH --output=test_%j.out    # Name standard output file

## Software
module purge                     # Purge all existing modules

## User commands
echo "This is a test scheduled by $USER"
```

Scheduling a batch job with your script

- Once a job script has been constructed you must schedule it on the HPC system using the SLURM `sbatch` command, e.g.:

```
sbatch /path/to/my_first_job.sh
```

Job output

- Once a job completes its execution, the standard output of the script will be redirected to an output file.
 - Great for debugging!
 - Could be different from output generated by your application
 - File is created in directory where job was run unless specified in your `--output` directive.
 - If the directive `--output` is not provided, then a generic file name will be used (`slurm_XXXXXX.out`).

Checking on your jobs

- **squeue**: Monitor status of **queued and running jobs**
 - By default, shows all jobs in queue can specify using:

```
$ squeue -u <username>  
$ squeue -p <partition>
```

- **sacct**: Check back on usage statistics of **previous jobs**
 - By default, only checks all jobs from the start of the current day can specify using:

```
$ sacct -u <username>  
$ sacct --start=MM/DD/YY -u <username>  
$ sacct -j <job-id>
```

Using software in jobs

- Okay so running a job is easy, but how do I use the software in my job?
 - We can utilize all the software we discussed in Module 4
 - Any non-graphics related commands you would normally run from the command line can simply be copied to your job script!
 - If you would like to use graphical applications we recommend running an interactive job in the “core desktop” app in OnDemand instead.

Software job script example

```
#!/bin/bash

## Directives
#SBATCH --nodes=1           # Number of requested nodes
#SBATCH --ntasks=1          # Number of requested tasks/cores
#SBATCH --time=0:01:00      # Max run time
#SBATCH --partition=acpu    # Specify Alpine CPU node
#SBATCH --qos=cpu-normal    # Specify QoS
#SBATCH --output=test_%j.out # Name standard output file

## Software
module purge                # Purge all existing modules
module load miniforge        # Load conda
conda activate <my-conda-environment> # Activate CONDA environment

## Run Python script
python my_cool_script.py
```

Interactive jobs

- Interactive jobs are used to gain real-time access to a compute node
 - Great for testing and debugging!
 - Interactive jobs can be subject to normal wait times!
- You can schedule an interactive job with `sinteractive`
- CURC also has a specialized command called `acompile` that provides you with an “instant” interactive session on Alpine
 - Limit of 4 CPUs for up to 12 hours

Running an interactive job

- Here we will run a Python script using an interactive job
- First request resources:

```
$ sinteractive --partition=acompile --qos=compile --time=00:10:00  
--nodes=1 --ntasks=1
```

- When the job starts you will be put on a compute node and you can execute your commands:

```
$ module load miniforge  
$ python my_very_cool_script.py
```

- To quit:

```
$ exit
```


Running an interactive graphical job

<https://ondemand-rmacc.rc.colorado.edu>

(Spawn a job with the “core desktop” application)

Documentation



<https://curc.readthedocs.io/en/latest/>